

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Arhitectură Agentică pentru Asistență Inteligentă în
Dezvoltarea Software**

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației***

Conducător științific
Ș.L. dr. ing. Mihai DOGARIU

Absolvent
Eduard-Daniel NANESCU

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **Eduard-Daniel NANESCU, 441A**

1. Titlul temei: Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software

2. Descrierea temei:

Contribuția originală a lucrării va consta în dezvoltarea unei arhitecturi software bazată pe Modele Mari de Limbaj (LLMs - Large Language Models), capabilă să asiste utilizatorul în scrierea, depanarea, documentarea și explicarea codului sursă. Aplicațiile vizate sunt eficientizarea procesului de dezvoltare software și automatizarea sarcinilor repetitive întâlnite în programare. Sistemul propus nu vizează antrenarea de la zero a modelelor, ci implementarea unei soluții agentice care utilizează modele pre-antrenate pentru a executa acțiuni complexe. Cercetarea aferentă va include: sinteza bibliografică a realizărilor actuale din domeniu, utilizarea unui mecanism care permite modelului să utilizeze instrumente pentru a interacționa cu mediul virtual, implementarea unor tehnici de inginerie a prompturilor și management-ul contextului pentru a asigura relevanța răspunsurilor, validarea experimentală a prototipului dezvoltat prin scenarii de utilizare reale utilizând medii de dezvoltare precum PyCharm/VS Code și biblioteci specifice precum LangChain, LangGraph sau GoogleADK, concluzionarea rezultatelor obținute și a contribuției originale, cât și enunțarea perspectivelor ulterioare de cercetare.

3. Contribuția originală:

Proiectarea și implementarea arhitecturii agentului, incluzând logica de selecție și utilizare a instrumentelor, gestionarea contextului și dezvoltarea unei interfețe sau integrarea cu interfețe existente care să permită interacțiunea facilă cu asistentul.

4. Resurse și bibliografie:

- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. "React: Synergizing reasoning and acting in language models." In The eleventh international conference on learning representations. 2022.
- Bozkurt, Aras, and Ramesh C. Sharma. "Generative AI and prompt engineering: The art of whispering to let the genie out of the algorithmic world." Asian Journal of Distance Education 18, no. 2 (2023): i-vii.
- Alto, Valentina. Building LLM Powered Applications: Create intelligent apps and agents with large language models. Packt Publishing Ltd, 2024.
- Zhu, He, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao et al. "Oagents: An empirical study of building effective agents." arXiv preprint arXiv:2506.15741 (2025).

5. Data înregistrării temei: 2025-12-11 22:54:00

Conducător(i) lucrare,
Ș.L. dr. ing. Mihai DOGARIU

Student,
Eduard-Daniel NANESCU

Director departament,
Conf. dr. ing Bogdan FLOREA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **621d9dfde3**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Eduard-Daniel NANESCU

.....

Cuprins

LISTA FIGURILOR	2
LISTA TABELELOR	3
LISTA DE ACRONIME	4
INTRODUCERE	5
CONTEXT SI MOTIVATIE	5
SCOPUL SI OBIECTIVELE LUCRARI	5
1. LUCRARI ANTERIOARE	7
1.1. Drumul catre modelele mari de limbaj (Large Language Model (LLM))	7
1.2. Maturizarea modelelor mari de limbaj si cercetari actuale	9
1.3. Solutii agentice pentru industria dezvoltarii software	10
1.4. Necesitati pentru dezvoltarea unei solutii agentice in domeniul dezvoltarii software	12
1.5. Puncte slabe ale arhitecturilor agentice	13
1.6. Alegerea solutiei de implementare pe baza lucrarilor studiate si a standardelor din industrie	14
2. IMPLEMENTARE	17
2.1. Mediul de dezvoltare	17
2.1.1. Cerinte hardware	17
2.1.2. Cerinte software	17
2.2. Structura	18
2.3. Configurare	18
2.4. Interfata cu utilizatorul si instructiuni de utilizare	19
BIBLIOGRAFIE	22

LISTA FIGURILOR

1.1.	Modelul unui perceptron	7
1.2.	Convolutia cu un nucleu a unei imagini	8
1.3.	Recurrent Neural Network (RNN) si Long Short Term Memory (LSTM) comparate	8
1.4.	Arhitectura transformer	9
1.5.	Evolutia modelelor in anii recenti [1]	10
1.6.	Evolutia modelelor GPT [1]	10
1.7.	Clasamentul modelelor in SWE-Bench Pro [2] la momentul actual in functie de performanta rezolvarii sarcinilor complexe in limba engleza	12
1.8.	Clasamentul modelelor in SWE-Bench Pro [2] la momentul actual in functie de cost	13
2.1.	Structura directoarelor proiectului	18
2.2.	Prima versiune a interfetei, realizata cu ajutorul libreriei Textual	19
2.3.	Continuarea conversatiei cu prima versiune a interfetei, realizata cu ajutorul libreriei Textual	19
2.4.	A doua versiune a interfetei, realizata fara libraria Textual	20
2.5.	Continuarea conversatiei cu a doua versiune a interfetei, realizata fara libraria Textual	20

LISTA TABELELOR

1.1.	Comparatia modelelor existente si a capabilitatilor acestora	11
1.2.	Comparație framework-uri pentru workflow-uri agentice	15
1.3.	Comparație biblioteci pentru interfața cu utilizatorul	16
2.1.	Hardware recomandat pentru modele open-source	17
2.2.	Comenzi disponibile in interfata	21

LISTA DE ACRONIME

API Application Programming Interface	MCP Model Context Protocol
BTT Backpropagation Through Time	OOS Out of Scope
CD Continuous Deployment	RAG Retrieval Augmented Generation
CI Continuous Integration	RCA Root Cause Analysis
CNN Convolutional Neural Network	RNN Recurrent Neural Network
E2E End to End	SDD Spec-Driven Development
IA Inteligență Artificială	SLM Small Language Model
LLM Large Language Model	TDD Test-Driven Development
LSTM Long Short Term Memory	TUI Terminal User Interface

INTRODUCERE

CONTEXT SI MOTIVATIE

Motivatia acestei lucrari vine din experienta personala in industria dezvoltarii software. Atributiile unui inginer software sunt multe, diverse si adesea repetitive si ingreuneaza procesul de dezvoltare. A scrie cod a devenit doar o mica parte din sarcinile intrebuintate unui inginer software, pierzand teren in fata design-urilor, documentatiei, review-urilor, a testelor unitare de integrare si End to End (E2E), instrumentarii si monitorizarii, procesului de Continuous Integration (CI)/Continuous Deployment (CD), infrastructurii, iar in cazul problemelor a debugging-ului si a analizei cauzelor principale (Root Cause Analysis (RCA)).

Un inginer software care lucreaza intr-o companie ce serveste milioane de clienti din intreaga lume trebuie sa fie la curent cu cele mai noi practici din domeniu, sa stie notiuni de programare, securitate cibernetica, networking si infrastructura, dar sa si retina simultan informatii vitale despre arhitectura si sistemele companiei, totul pe langa instruirea colegilor noi sau sedinte.

Timpul unui inginer este astfel impartit de foarte multe procese dinamice care trebuiesc adresate de un angajat cu experienta.

Avand in vedere cele de mai sus, cat si evolutia recenta a sistemelor bazate pe Inteligență Artificială (IA) ce folosesc un Large Language Model (LLM) pentru intelegerea si generarea de text, era natural ca directia in care dezvoltarea software urma sa se schimbe catre un proces in care multe din sarcinile pe care un inginer le-ar fi avut sunt acum delegate sistemelor automate.

Datorita faptului ca aceste modele functioneaza cu text ca date de intrare iar programarea este realizata de asemenea folosind text, nu a fost dificil sa se treaca de la o experienta pur conversationala cu aceste modele la intrebuintari pentru cautare prin limbaj natural, iar in final integrandu-se cu mediile de dezvoltare software si cu terminalul sistemului de operare pentru a avea acces la un context mai bun dar si pentru executia de comenzi putand astfel ca un model de IA sa interactioneze cu mediul inconjurator.

SCOPUL SI OBIECTIVELE LUCRARI

Scopul acestei lucrari este de a realiza un asistent extensibil bazat pe IA, agnostic fata de platforma, care poate functiona cu o multitudine de modele de tipul LLM sau Small Language Model (SLM), ce ruleaza local sau in cloud si ce poate interactiona cu mediul inconjurator (sistem de operare, mediu de dezvoltare, internet) prin intermediul uneltelor (tool-uri ¹), realizat de un workflow agentic ²

Obiectivul principal care trebuie atins este realizarea unui produs functional ce poate ajuta un inginer software in cat mai multe aspecte ale meseriei sale. Pentru a reusi sa indeplinim acest obiectiv, trebuie sa avem in vedere urmatoarele necesitati: interfata facila ce poate fi folosita

¹In IA folosita cu ajutorul agentilor **tool**-urile sunt functii cu descrieri detaliate de folosire pe care agentul le populeaza cu parametrii, le executa si receptioneaza raspunsul pentru a-l folosi mai departe in rezolvarea problemelor complexe

²**Agentii** sunt aplicatii ce folosesc inteligenta artificiala pentru a interactiona cu mediul inconjurator

oriunde folosind terminalul sistemului de operare (Terminal User Interface (TUI)), suficiente unelte pentru a indeplini sarcini cat mai complexe, management-ul conversatiei, contextului si al memoriei, versionare, evaluari si teste de regresie, monitorizare si observabilitate (metrici), siguranta (code execution sandboxes pentru executia de cod, guardrails, Out of Scope (OOS) detection), si cel mai important: un system prompt ales cu grija.

Doua obiective pe care aceasta lucrare nu le prezinta sunt acceptarea de date de intrare multimodale (imagini, documente, videoclipuri) si punerea in piata a acestui produs.

Capitolul 1

LUCRARI ANTERIOARE

O buna intelegere a tehnologiilor, standardelor si a obstacolelor actuale din industrie necesita studierea literaturii de specialitate, a lucrarilor si a arhitecturilor existente deja in domeniu. Acest pas este obligatoriu pentru alegeri informate in cadrul implementarii proiectului, evitarea problemelor deja recunoscute si confera o vedere de ansamblu asupra stadiului final al lucrarii.

1.1 Drumul catre modelele mari de limbaj (LLM)

Domeniul inteligentei artificiale a fost studiat o perioada indelungata iar cateva publicatii au marcat industria prin concepte revolutionare si progrese importante.

Unul din conceptele de baza ale inteligentei artificiale a aparut pentru prima data publicat in anul 1958 de catre F. Rosenblatt in [3], unde ni se prezenta perceptronul: o replica a unui neuron biologic capabil sa invete clasificare binara. Modelul invata prin ajustarea ponderilor proprii in functie de eroarea produsa la iesire, formand baza invatarii artificiale. Desi revolutionar, aceasta abordare restrictiona la o singura dimensiune datele de intrare.

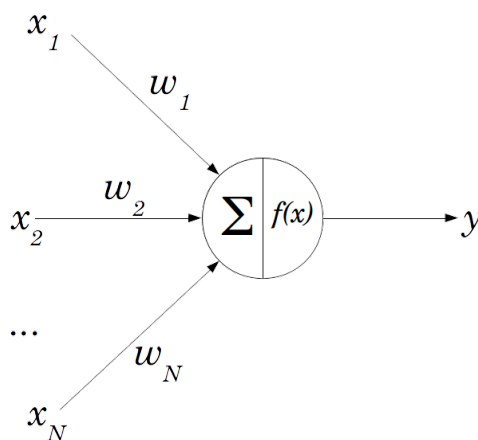


Figura 1.1: Modelul unui perceptron

Schimbarea a venit cand Yann LeCun si echipa sa au introdus in [4] retelele neurale convolutive (Convolutional Neural Network (CNN)), un concept care planuia sa rezolve aceasta problema. Au aparut astfel primele aplicatii ale inteligentei artificiale in domeniul imaginilor. Ideea de baza era simpla: in loc de datele unidimensionale caracteristice perceptronului, acum modelul folosea "ferestre" ale unei imagini, mai multe straturi cat si operatii speciale pentru a invata caracteristici spatiale complexe.

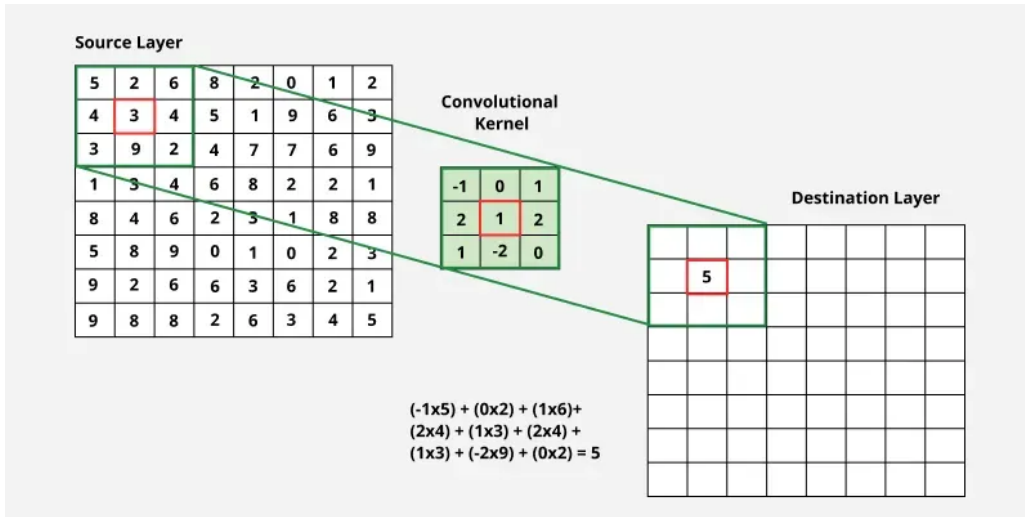


Figura 1.2: Convolutia cu un nucleu a unei imagini

Desi CNN sunt foarte bune in a procesa date statice, alte aplicatii precum procesarea limbajului nu pot fi incadrate intr-o "fereastră". Vorbirea si textul sunt secventiale, deci se intampla in timp. Incercarile initiale in a infrunta aceasta problema se folosind Recurrent Neural Network (RNN) se loveau de o problema fundamentala a invatarii artificiale: disparitia gradientilor. Datorita conceptului de Backpropagation Through Time (BTT), in cadrul unor secvente foarte lungi, gradientii deveneau atat de mici incat ajungeau sa fie irelevanti, cauzand modelul sa uite date de la inceputul secventei. Progresul a fost atins cand Hochreiter and Schmbithuber au introdus conceptul de Long Short Term Memory (LSTM) in [5] care a dat inteligentei artificiale o memorie functionala. Prin conceptul de "porti", retea reusea sa controleze ce informatie trebuie sa fie uitata si care trebuie sa fie pastrata.

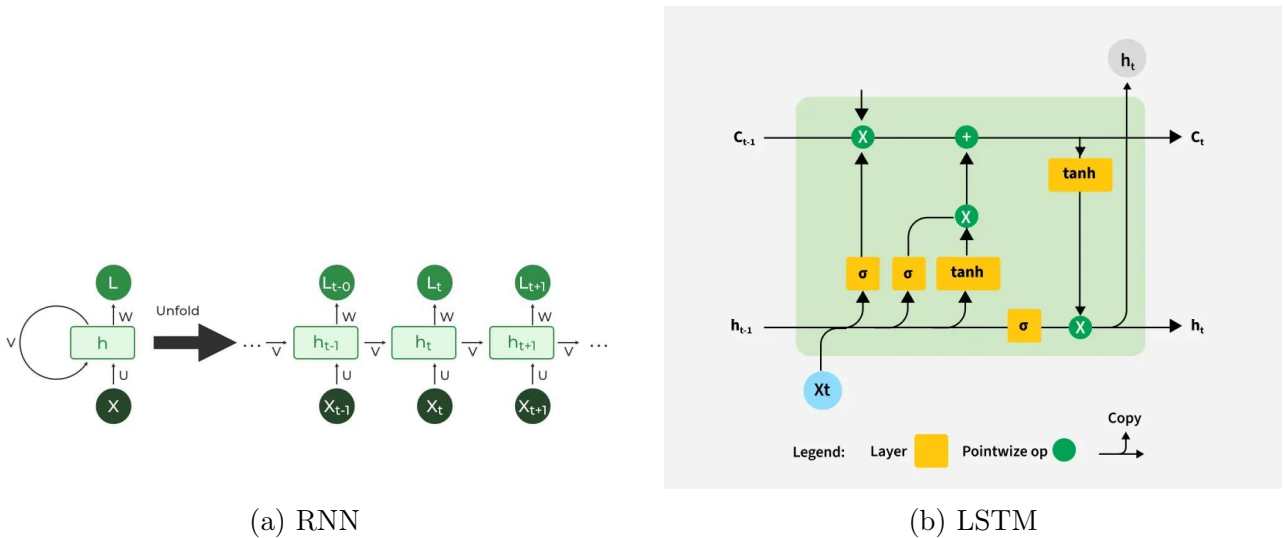


Figura 1.3: RNN si LSTM comparate

Urmatorul mare pas pentru industria urma sa fie arhitectura transformer prezentata de Vaswani et al. in celebra lucrare [6]. Arhitectura LSTM trebuia sa proceseze pas cu pas o secventa, iar aceasta limitare facea antrenarea lor pe o cantitate mare de date foarte ineficienta. Arhitectura transformer utilizeaza o tehnica numita "self-attention" pentru a se uita la fiecare cuvânt dintr-o secventa simultan, calculand astfel legatura dintre acestea. Odata disparuta limitarea din punctul de vedere al vitezei de antrenare, s-a deschis poarta antrenarii pe baze de date uriase, lucru care a dat nastere modelelor mari de limbaj.

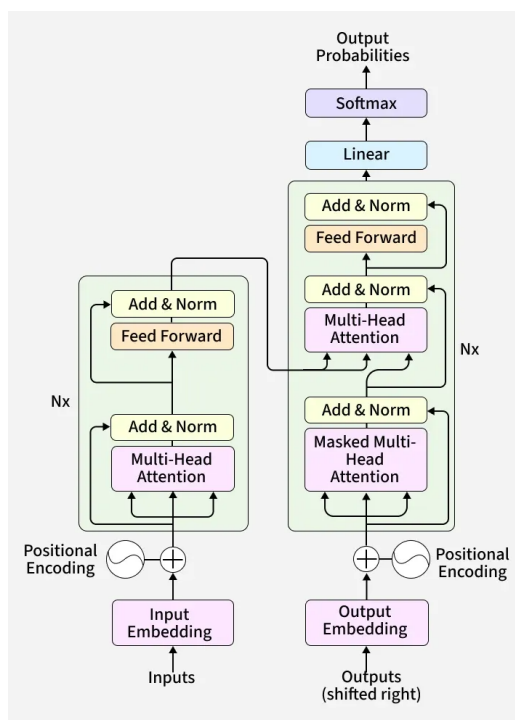


Figura 1.4: Arhitectura transformer

Modelele mari de limbaj sunt produsul antrenarii pe cantitati masive de date sustrate din intreg internetul. Vazand ca daca continuau sa introduca si mai multe date de antrenare, cercetatorii de la OpenAI performanta modelului nu parea sa scada, au reusit sa introduca 175 de miliarde de parametrii in modelul GPT-3, iar descoperirile au fost relatate in lucrarea "Language Models are Few-Shot Learners" [7]. Marea realizare a fost ca fara o instruire prealabila modelul a invatat singur cum sa rezolve sarcini de programare si sumarizare a textului, poezie dar si raspunsuri punctuale la intrebari. Inteligenta artificiala a facut pasul de la intrebutari specializate la proficianta intr-o multitudine de domenii.

1.2 Maturizarea modelelor mari de limbaj si cercetari actuale

Odata cu aparitia primelor foundation models ¹ precum GPT-3 sau GPT-4, Sonnet sau Opus, industria inteligentei artificiale a explodat. Aplicatii de toate tipurile au inceput sa foloseasca aceste modele pentru diverse sarcini: programare, traduceri sau experiente conversationale. Odata cu cererea, companiile au investit masiv in cercetare si centre ce pot antrena modele mai puternice. Astfel, in fiecare an au aparut modele din ce in ce mai performante. Marii jucatori in domeniul acestor modele, printre care si OpenAI, Anthropic, Google si Meta se intrec in a scoate cel mai bun model.

¹Un foundation model este un model mare de limbaj de uz general antrenat pe cantitati foarte mari date diverse, ce poate fi adaptat pentru un numar mare de intrebutari

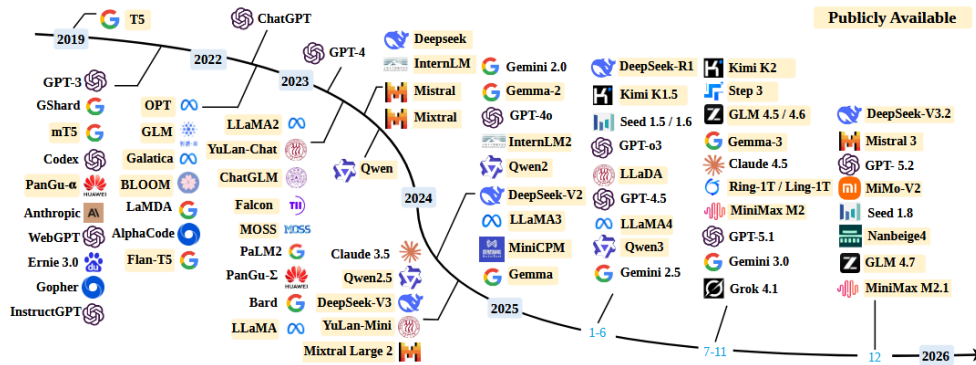


Figura 1.5: Evolutia modelelor in anii recenti [1]

Multe dintre evolutiile facute in anii recenti au fost facute in capabilitatea de a gandi mai adanc, marimea contextului, pre-antrenarea pe anumite date si integrarea cu aplicatii sau Application Programming Interface (API)-uri terțe. Un exemplu de evolutie al unui model fundational poate fi regasit in figura de mai jos:

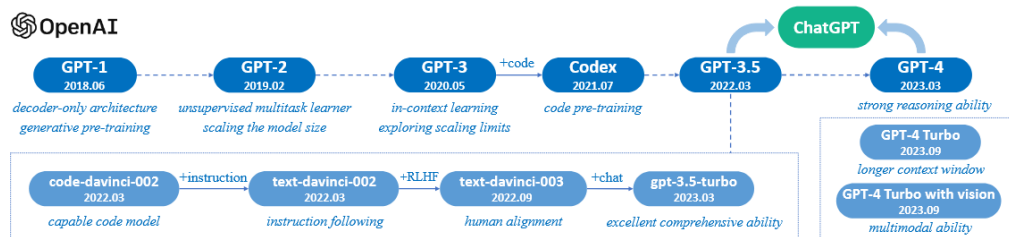


Figura 1.6: Evolutia modelelor GPT [1]

1.3 Solutii agentice pentru industria dezvoltarii software

Arhitecturile agentice presupun inzeestrarea modelelor mari de limbaj cu unelte necesare interactionarii cu ecosistemul in care ruleaza. Odata cu implementarea procedurii "Chain of Thought" [8] care permite navigarea problemelor complexe de catre un model de limbaj, generarea de planuri si cu abordarea ReAct [9] care impreuna incearca sa combata fenomenul halucinatiilor ² si inconsistenta in raspunsuri cu care se confruntau primele modele mari de limbaj.

Increderea in procesul de gandire al modelelor de inteligenta artificiala s-a cimentat, incepand sa produca rezultate consistente, iar astfel a urmat inzeestrarea acestora cu unelte care sa le permita interactiunea cu mediul in care ruleaza. Numarul uneltelor ce le ofera capabilitatea de a cauta pe internet pentru a gasi date actuale si a nu se limita la datele de antrenare, la posibilitatea rularii de comenzi in terminalul sistemului de operare sau chiar preluarea controlului browserului personal pentru automatizari, a crescut exponential, si odata cu el si numarul de aplicatii ale acestor modele.

Industria dezvoltarii software are probabil una din cele mai profitabile intrebuintari datorita volumului masiv de munca existent in domeniu, asa ca nu a durat mult pana ce marii jucatori

²Fenomenul de halucinatii in modelele de limbaj se refera la generarea unor raspunsuri care nu sunt ancorate in fapte reale sau care nu au legatura cu prompt-ul de intrare

pe piata modelelor de IA au inceput sa ofere produse care sa asiste programatorii in sarcinile lor. La momentul actual exista mai multe solutii competitive: Codex de la OpenAI, Gemini CLI de la Google sau Claude Caude de la Anthropic. Toate acestea sunt solutii proprietare care sunt facute sa ruleze cu modelele proprii si pentru care utilizatorul trebuie sa plateasca. Exista si solutii open-source precum precum OpenCode, Kiro sau OpenClaw cu care detinatorii de modele se lupta pentru a domina piata, desi lupta nu este stransa din cauza monopolului solutiilor proprietare.

Companie	Model	Pret [\$/MTok]	Capabilitati
Anthropic (link)	Claude Opus 4.6	\$5/input MTok, \$25/output MTok	Lent, cel mai inteligent, gandire adaptiva, 1M fereastră de context, 128k tokens max. la iesire
	Claude Sonnet 4.6	\$3/input MTok, \$15/output MTok	Mai rapid, compromite niste inteligenta pentru performante, gandire adaptiva, 1M fereastră de context, 64k tokens max. la iesire
	Claude Haiku	\$1/input MTok, \$5/output MTok	Cel mai rapid, fara gandire adaptiva, 200k fereastră de context, 64k tokens max. la iesire
OpenAI (link)	gpt-5.4	\$2.5/input MTok, \$15/output MTok	Cel mai inteligent pentru sarcini agentice, de programare si profesionale. 1M fereastră de context si 128k tokens max. la iesire
	gpt-5.4-mini	\$0.75/input MTok, \$4.5/output MTok	Model mai mic, rapid dar puternic pentru programare si subagenti. 400k fereastră de context si 128k tokens max. la iesire
	gpt-5.4-nano	\$0.20/input MTok, \$1.25/output MTok	Model mic si foarte rapid pentru sarcini usoare in volum mare. 200k fereastră de context si 128k tokens max. la iesire
Google (link)	Gemini 3.1 Pro	\$2/input MTok, \$12/output MTok < 200k tokens, \$4/input MTok, \$118/output MTok >= 200k tokens	Model pentru sarcini complexe ce necesita cunostiinte dezvoltate. Fereastră de context 1M in / 64k out
	Gemini 3.1 Flash	\$0.5/input MTok, \$3/output MTok	Inteligenta comparabila cu modelul Pro la o viteza mai mare. Fereastră de context 1M in / 64k out
	Gemini 3.1 Flash-Lite	\$0.25/input MTok, \$1.5/output MTok	Model construit pentru costuri reduse si volum mare de sarcini. Fereastră de context 1M in / 64k out
	Gemma 4	Open Source	Familie de modele open-source pentru rulare locala, cost redus si personalizare
DeepSeek (link)	Deepseek Chat 3.2	\$0.028 sau \$0.28/input MTok, \$0.42/output MTok	Mod fara gandire adanca, 128k fereastră de context, max. out token: 8k
	Deepseek Reasoner 3.2	\$0.028 sau \$0.28/input MTok, \$0.42/output MTok	Mod cu gandire adanca, 128k fereastră de context, max. out token: 64k
Meta (link)	Llama 4	Open Source	Performante similare cu GPT4-o sau Gemini 2.0 Flash, dar open-source. Fereastră de context 10M/1M in functie de model.

Tabela 1.1: Comparatia modelelor existente si a capabilitatilor acestora

Nota: Toate modelele au si versiuni pentru diferite nivele de efort pentru gandire (reasoning effort) sau pentru date multimodale ce pot influenta si costul dar si rezultatele

Din punct de vedere al puterii de rezolvare ale sarcinilor reale din industria dezvoltarii software, clasamentul celor mai bune modele se schimba frecvent. Utilizand SWE-Bench Pro [2], putem masura puterea de abordare a acestor modele de top contra unor probleme reale.

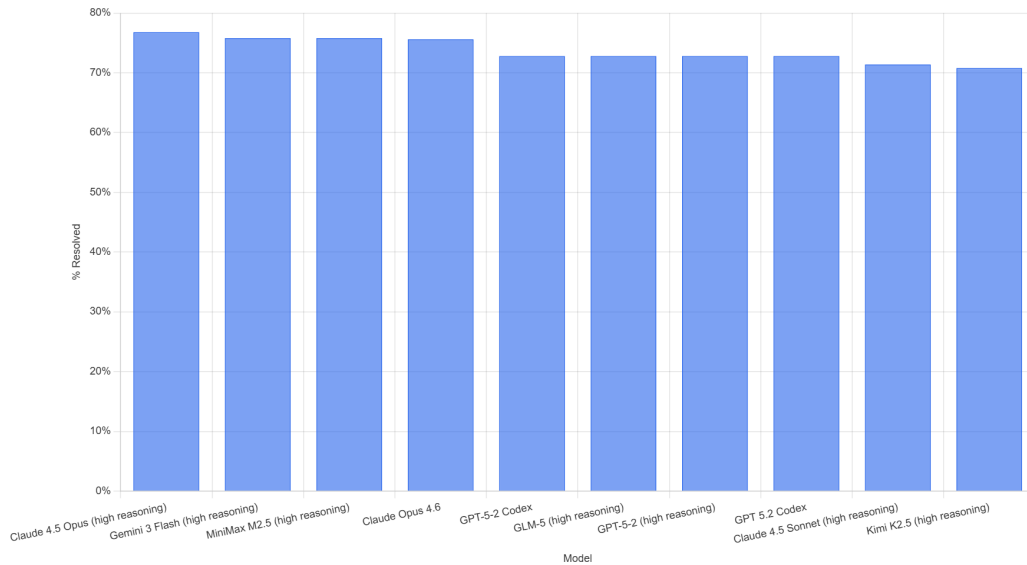


Figura 1.7: Clasamentul modelelor in SWE-Bench Pro [2] la momentul actual in functie de performanta rezolvarii sarcinilor complexe in limba engleza

Pe langa puterea de rezolvare, un alt fenomen a aparut care schimba rata de adoptie a unui model: costul. Desi performantele modelelor cresc si acestea sunt din ce in ce mai integrate in vietile noastre de zi cu zi, asemenea telefoanelor mobile, costurile de rulare ale acestor modele creste si el. Cele mai puternice modele au nevoie de foarte multe resurse pentru antrenare dar si pentru inferenta. Acest lucru se reflecta asupra utilizatorului final care trebuie sa plateasca per-token ³ sau un abonament lunar care include un numar limitat de tokens. Modelele cu capacitati de gandire adanca, pe langa faptul ca au un cost mai mare per-token, consuma si mai multi tokeni din cauza gandirii dezvoltate sau a ferestrei de context mari, ridicand si mai mult costurile. Pentru asta, fiecare detinator de modele fundamentale, are mai multe versiuni ale aceluasi model, pentru intrebuintari de diverse complexitati, pentru a ajuta cu costurile dar si cu balansarea resurselor. Spre exemplu Google detine modelele din gama Gemini, cu versiunile "Flash", "Thinking", "Pro".

Avand in vedere problema costului dar si a balansarii traficului, aplicatiile de asistenti de programare de pe piata, au implementat o modalitate de a clasifica prompt-urile utilizatorului in functie de complexitate si rutarea acestui prompt catre un model potrivit sa rezolve aceasta problema si optim din punct de vedere al costului.

1.4 Necesitati pentru dezvoltarea unei solutii agentice in domeniul dezvoltarii software

Pentru a realiza un asistent inteligent in domeniul dezvoltarii software este nevoie sa atingem cateva obiective principale, printre care: o interfata accesibila din diferite medii de dezvoltare pentru a satisface cat mai multe nevoie, unelte ce pot rula in terminalul oricarui sistem de ope-

³Token-ul este un cuvant sau o parte din cuvant si este elementul fundamental procesat de modelele mari de limbaj

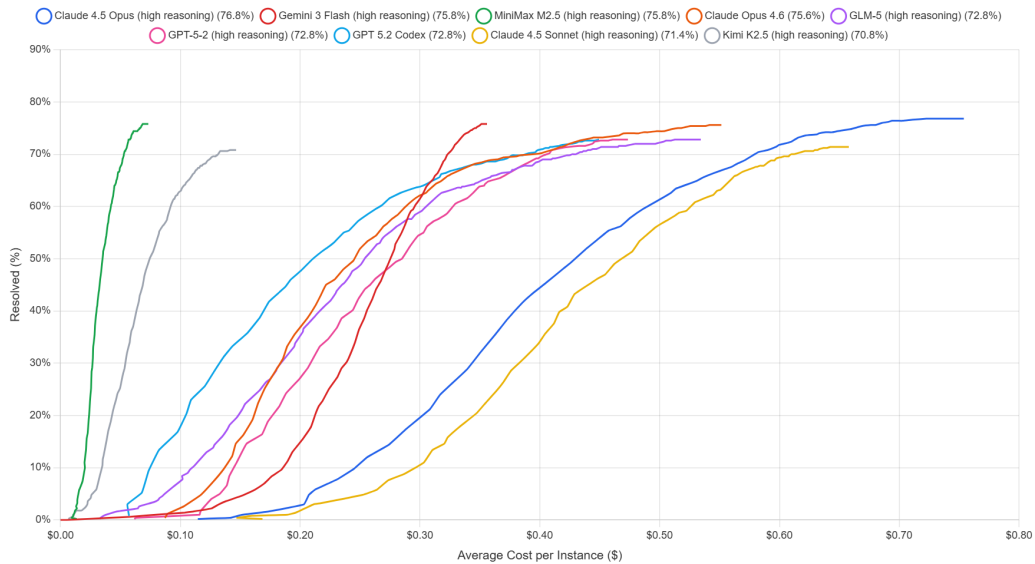


Figura 1.8: Clasamentul modelelor in SWE-Bench Pro [2] la momentul actual in functie de cost

rare, posibilitatea de a alege dintr-o sumedenie de modele, comenzi pentru a controla parametrii modelelor si un prompt de sistem si skill-uri specializate pentru operatii din domeniu.

Agentii functioneaza, pe langa procedeele expuse anterior, pe una dintre aceste filozofii de programare: Spec-Driven Development (SDD) sau Test-Driven Development (TDD). Primul se refera la programarea pe baza unor documente de specificatie ce expun in detaliu toate necesitatile implementarii, iar cel din urma se refera la programarea dupa cerintele sumare si executarea testelor la final pentru a confirma comportamentul dorit.

Pe langa aceste obiective de baza, trebuie sa ne asiguram ca modelul raspunde doar prompt-urilor cu scop legitim, si sa ne asiguram ca modelul nu poate rula comenzi care pun in pericol integritatea sistemului pe care ruleaza sau compromite date sensibile, lucruri pe care le vom discuta in capitolele ce urmeaza.

1.5 Puncte slabe ale arhitecturilor agentice

Desi entuziasmul pentru bula IA atinge cote maxime, inca este mult loc de imbunatarie, iar aceste platforme de asistenti de programare au inca multe defecte pe care trebuie sa le depasasca.

In primul rand, bariera principala in avansarea acestor asistenti este fereastra de context. Aceasta fereastră reprezinta cata informatie poate sa tina minte agentul. Pentru sisteme dezvoltate cu zeci de mii de linii de cod, sau solutii impartite pe micro servicii in mai multe proiecte, acest context limiteaza masiv capabilitatile agentilor. Exista multe tehnici de optimizare a acestui context de la compresie, realizarea unor fisiere ce rezuma cunostiintele actuale si inceperea unei conversatii noi care pleaca de la acel rezumat pana la rularea de cod pentru actiuni repetitive sau alegerea cu grija a prompt-ului. Dar din pacate toate aceste tehnici nu sunt de ajuns. Multe functionalitati care fac un agent suficient de performant, precum tool-urile sau Retrieval Augmented Generation (RAG) [10] reusesc sa consume rapid aceasta fereastră de context. Poate 200.000 de tokeni pot parea un numar mare, dar defapt chiar si acest numar, precum la memoria hardware este defapt mai mic. Din aceasta fereastră doar 160.000 de tokens sunt disponibili cu adevarat. O parte din ei sunt rezervati pentru auto-compactare sau pentru iesire. Depasirea acestei ferestre de context scade astfel performantele agentului si se recomanda inceperea unei conversatii noi.

In al doilea rand, modelele curente par sa genereze date de iesire mai detaliate decat este

necesar, preferand calitatea peste cantitate. De multe ori in sarcini de tipul "PR Review", agentii se grabesc sa relateze multe probleme simple sau de formatare si rateaza probleme de logica.

In al treilea mod, similar cu punctul anterior, agentii nu au o buna intelegere legata de cand ar trebui sa opreasca investigatiile sau sa incerce alte solutii si raman blocati in bucle infinite de a incerca aceleasi lucruri si a esua.

Pentru toate aceste probleme se lucreaza in mod curent la rezolvare de care vom vorbi ulterior.

1.6 Alegerea solutiei de implementare pe baza lucrarilor studiate si a standardelor din industrie

Solutia finala include mai multe categorii care trebuiesc comparate independent.

Pentru inceput, modelele disponibile initial vor fi din gama Gemini, Claude si OpenAI, cu posibilitatea de extindere la modele open-source rulate local sau in cloud.

Pentru partea de framework agentic, ofertele sunt numeroase si variate. Fiecare dintre optiunile de mai jos are avantaje si dezavantaje care trebuiesc cantarite in contextul aplicatiei noastre.

In materie de limbaj de programare, Python a fost alegerea evidenta deoarece este aproape folosit in totalitate in astfel de aplicatii, devenind un standard in industrie. Ecosistemul este de asemenea foarte matur si exista o sumedenie de biblioteci pentru a ne ajuta in impelmentare. De asemenea inca un atu este portabilitatea cu care vine, Python putand fi instalat cu usurinta pe orice sistem de operare cu o simpla comanda.

Am pornit de la ideea ca trebuie sa avem nevoie de un framework gandit pentru utilizat de unelte cu suport pentru mai multe modele, care sa suporte mai multe ramuri de executie, sa incorporeze managementul starii si al memoriei si sa poata construi un workflow robust si consistent. Pentru asta, am adunat o colectie de framework-uri folosite in industrie pentru a le compara si a decide care este cel potrivit.

In final am ales sa folosesc LangGraph datorita controlului mare pe care il ofera asupra desfasurarii workflow-ului agentic si a integrarii cu LangSmith ⁴ si LangFuse ⁵ dar si pentru compatibilitatea cu logica desfasurarii procesului de asistenta in dezvoltarea software, respectiv buclele de scriere de cod, feedback, investigare si imbunatatire, in ciuda complexitatii mai mari pe care o constituie acest framework.

⁴Unealta de observabilitate integrata cu LangGraph si nu numai

⁵Unealta de observabilitate similara cu LangSmith dar open-source

Tabela 1.2: Comparație framework-uri pentru workflow-uri agentice

Criteriu	LangChain	LangGraph	Google Gen AI / Vertex	LiteLLM	OpenAI SDK
Rol Principal	Orchestrator general de LLM-uri și integrări.	Framework pentru agenți stateful și fluxuri ciclice.	Platformă enterprise integrată în Google Cloud.	Proxy sau router universal pentru 100+ LLM-uri.	Client oficial pentru modelele OpenAI.
Nivel Abstr.	Înalt (multe componente gata de utilizat).	Mediu/Înalt (control fin asupra grafurilor).	Înalt (Managed Services în Vertex AI).	Jos (strat de traducere API).	Jos/Mediu (interacțiune directă).
Capabilități Agentice	Limitate (predominant fluxuri liniare).	Excelent (bucle, feedback și multi-agent).	Bun (suport nativ pentru tool-uri).	N/A (doar rutare de mesaje).	Depinde de utilizator (manual sau Assistants API).
Suport Multi-Model	Da (agnostic).	Da (prin ecosistemul LangChain).	Optimizat pentru Gemini.	Absolut (sco-pul principal).	Nu (doar OpenAI).
Management Stare	Încorporat (Memory buffers).	Superior (persistență și „time travel”).	Gestionat de platformă.	N/A (stateless).	Threads (doar în Assistants API).
Învățare	Medie (ecosistem foarte mare).	Abruptă (necesită logica grafurilor).	Ușoară (UI intuitiv în Cloud).	Ușoară (identic cu OpenAI SDK).	Foarte Ușoară, documentat

Pentru interacțiunea cu utilizatorul, ideea de pornire a fost ca trebuie să avem o interfață care poate fi rulată în orice context. Gândul m-a purtat automat către un TUI deoarece orice sistem de operare detine un terminal care poate fi pornit în orice mediu de programare, oferind o gamă largă de utilizatori potențiali. Mai departe am constituit o listă de abordări și librării pe care le putem folosi pentru a construi această interfață, fiecare cu avantaje și dezavantaje care trebuie comparate.

Tabela 1.3: Comparație biblioteci pentru interfața cu utilizatorul

Bibliotecă	Arhitectură	Avantaje	Dezavantaje
Textual	Motor TUI pe ecran complet folosind buffer-ul alternativ al terminalului. Arbore de widget-uri bazat pe DOM (similar cu HTML)	• Sistem de layout similar cu CSS (grid, dock) • Formatare de text încorporată nativ • Complet asincron și bazat pe evenimente	• Folosește un scrollbar fals (nu funcționează scroll-ul nativ) • Supraîncărcarea DOM-ului cauzează lag dacă mesajele vechi nu sunt șterse • Dificultate la copierea/lipirea nativă a textului
Prompt Toolkit și Rich	Experiență interactivă folosind buffer-ul principal al terminalului. Afișare prin stdout standard.	• Derulare perfectă prin scrollbar-ul nativ al terminalului • Zero lag sau probleme de memorie pe măsură ce istoricul crește • Selectarea și căutarea nativă a textului funcționează impecabil	• Nu este un motor real de layout UI (doar flux liniar) • Dificil de implementat panouri laterale sau complexe • Limitat strict la input ancorat în partea de jos
Urwid	TUI clasic pe ecran complet folosind buffer-ul alternativ al terminalului. Ierarhie strictă de widget-uri.	• Extrem de matur, stabil și testat în producție • Randare foarte performantă pentru liste mari • Gestionare excelentă a input-ului standard din terminal	• API învechit și curbă de învățare mai abruptă • Lipsește stilizarea modernă încorporată • Sincron implicit (necesită integrare suplimentară)

Initial, am ales sa folosesc abordarea cu Textual pentru ca ofera un sistem de layout foarte puternic, care poate fi folosit pentru a construi o interfata complexa, cu panouri laterale, tab-uri si meniuri. Insa, in timpul implementarii a trebuit sa abandonez aceasta idee deoarece am intampinat anumite probleme pe care o sa le dezvoltam in capitolul de implementare.

Astfel, am finalizat lista alegerilor arhitecturale principale, vom motiva si restul alegerilor punctual, pe parcursul implementarii.

Capitolul 2

IMPLEMENTARE

2.1 Mediul de dezvoltare

Expunerea elementelor constitutive disponibile care alcatuiesc mediul hardware cat si software este imperativa pentru relatarea procesului de dezvoltare si reproducerea pasilor prin care s-a ajuns la produsul final.

2.1.1 Cerinte hardware

Cerintele hardware pentru dezvoltarea si rularea acestei aplicatii sunt minime. In cazul in care folosim modelele din cloud este necesar doar un laptop sau computer ce poate rula Python 3 cu conexiune activa la internet. Deoarece inferenta nu este realizata local, nu este necesar un GPU performant sau o cantitate mare de RAM. Daca se doreste rularea unui model local, cerintele hardware depind de modelul ales, dar cresc exponential necesitand acces la o placa video moderna si o cantitate semnificativa de RAM.

Tabela 2.1: Hardware recomandat pentru modele open-source

Model	Parametri	GPU Recoman-dat	VRAM	RAM
LLaMA 3	70B	2x NVIDIA A100 80GB sau 2x RTX 4090	40–160 GB (cuan-tizat vs FP16)	64–128 GB
Mistral	7B	NVIDIA RTX 3060 / RTX 4090	4–16 GB	8–32 GB
Qwen 2.5 (14B)	14B	NVIDIA RTX 3090 / RTX 4090	16–24 GB	16–64 GB
Gemma 4 (E4B)	4.5B efectivi (8B total)	NVIDIA RTX 3060 / RTX 4060	4–16 GB (cuanti-zat vs BF16)	8–32 GB

2.1.2 Cerinte software

Cerintele software pentru rularea acestei aplicatii sunt minime de asemenea. Este necesar doar un sistem de operare cu terminal precum Windows, Linux sau macOS, cu Python 3 instalat si acces la internet.

În ceea ce privește mediul de dezvoltare orice mediu de dezvoltare cu suport pentru Python este suficient, iar singurul program necesar este `uv`¹ pentru a instala dependențele necesare.

2.2 Structura

Structura aplicației este una simplă și include directoare pentru documentație, cod sursă, interfață și fișiere de configurare. Codul este împărțit în două directoare principale `src` pentru codul sursă al agentului propriu zis și `ui` pentru codul sursă al interfeței cu utilizatorul. Documentația este organizată în directorul `docs` și în fișierul `README.md`, iar fișierele de configurare și de management al dependențelor sunt în folderul principal al proiectului.

```
vibe-cli/
|- docs/
| |- api/
| | |- postman_collection.json
| | \- swagger.yaml
| \- USAGE.md
|- quality/
|- scripts/
|- src/
| |- __init__.py
| \- main.py
|- ui/
| |- __init__.py
| |- app.py
| |- app_old.py
| |- commands.py
| \- ui.py
|- Makefile
|- README.md
|- config.yml
|- pyproject.toml
\ - uv.lock
```

Figura 2.1: Structura directoarelor proiectului

2.3 Configurare

Configurarea aplicației se face prin intermediul fișierului `config.yml` care conține toate setările necesare pentru a rula aplicația, precum cheile de API pentru modelele de inteligență artificială. Pentru a rula aplicația, utilizatorul trebuie să populeze variabilele declarate în fișierul de configurare cu cheile API corespunzătoare pentru fiecare provider. În cazul în care nu este configurată nici o cheie validă, aplicația nu va putea trimite request-uri către LLM și va atentiona utilizatorul că trebuie să populeze una dintre acele valori.

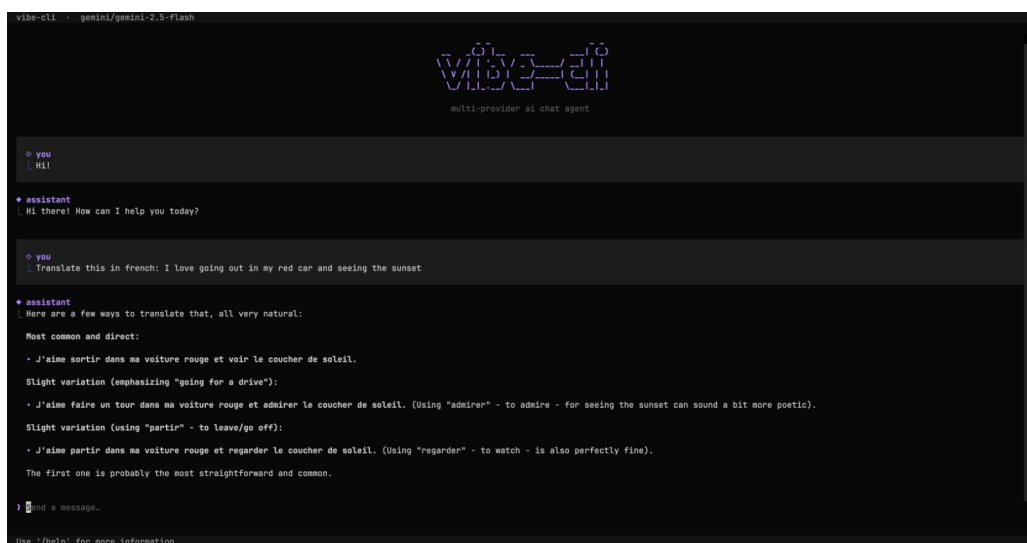
Câteva setări sunt de asemenea disponibile direct în interfața cu utilizatorul, precum selectarea modelului (dacă au fost introduse cheile pentru acesta), selectarea efortului de gândire, gestionarea conversațiilor și încă câteva.

¹`uv` este un manager de pachete pentru Python, similar cu `pip`, dar mult mai eficient

2.4 Interfata cu utilizatorul si instructiuni de utilizare

Interfata serveste ca un mediu de legatura intre utilizator si agent, dar si ca un spatiu de configurare al aplicatiei. Acesata apeleaza functii ale agentului de la distanta, putand rula si cu un agent local sau cu unul in cloud.

In dezvoltarea interfetei, am pornit folosind solutia oferita de biblioteca Textual. La finalul primului prototip interfata arata modern si functiona corect. Insa, pe masura ce am populat conversatia cu asistentul, am observat ca functionalitatea de scroll a ferestrei nu functiona corect, sarind catre partea de sus a ecranului. Dupa ce m-am interesat mai atent la solutiile implementate de alte programe asemanatoare precum Claude Code, am observat ca acestea nu folosesc un TUI propriu zis, ci doar scriu in terminal folosind stdout, lasand astfel scroll-ul nativ al terminalului sa se ocupe de istoricul conversatiei. Am decis astfel sa abandonez solutia cu Textual si sa implementez o interfata simpla care sa scrie in terminal folosind print-uri, oferind astfel o experienta mult mai fluida si fara probleme de scroll sau lag.



```
vibe-cli - gemini/gemini-2.5-flash

vibe-cli
multi-provider ai chat agent

> you
Hi!

* assistant
Hi there! How can I help you today?

> you
Translate this in french: I love going out in my red car and seeing the sunset

* assistant
Here are a few ways to translate that, all very natural:

Most common and direct:
- J'aime sortir dans ma voiture rouge et voir le coucher de soleil.

Slight variation (emphasizing "going for a drive"):
- J'aime faire un tour dans ma voiture rouge et admirer le coucher de soleil. (Using "admirer" - to admire - for seeing the sunset can sound a bit more poetic).

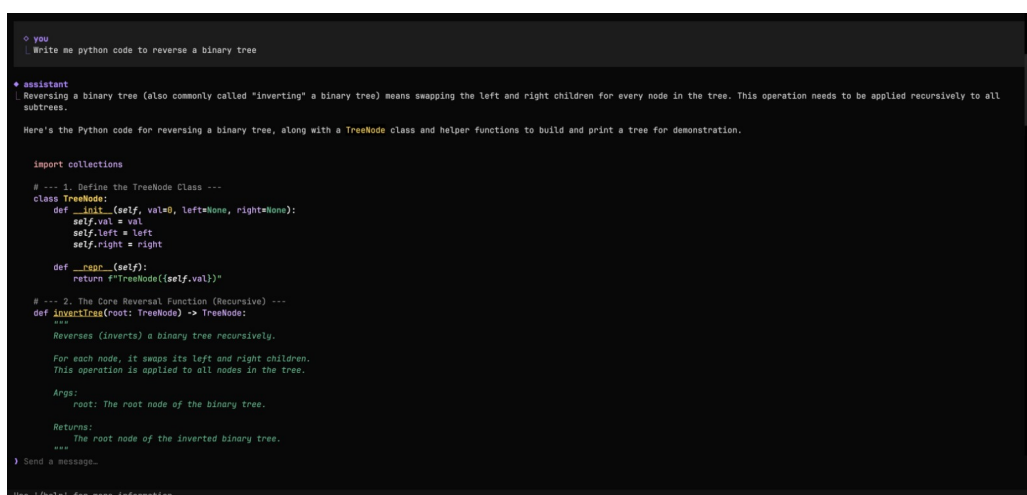
Slight variation (using "partir" - to leave/go off):
- J'aime partir dans ma voiture rouge et regarder le coucher de soleil. (Using "regarder" - to watch - is also perfectly fine).

The first one is probably the most straightforward and common.

) Send a message...

Use '/help' for more information
```

Figura 2.2: Prima versiune a interfetei, realizata cu ajutorul libreriei Textual



```
> you
Write me python code to reverse a binary tree

* assistant
Reversing a binary tree (also commonly called "inverting" a binary tree) means swapping the left and right children for every node in the tree. This operation needs to be applied recursively to all subtrees.

Here's the Python code for reversing a binary tree, along with a TreeNode class and helper functions to build and print a tree for demonstration.

import collections

# --- 1. Defining the TreeNode Class ---
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self):
        return f"TreeNode({self.val})"

# --- 2. The Core Reversal Function (Recursive) ---
def invertTree(root: TreeNode) -> TreeNode:
    """
    Reverses (inverts) a binary tree recursively.

    For each node, it swaps its left and right children.
    This operation is applied to all nodes in the tree.

    Args:
        root: The root node of the binary tree.

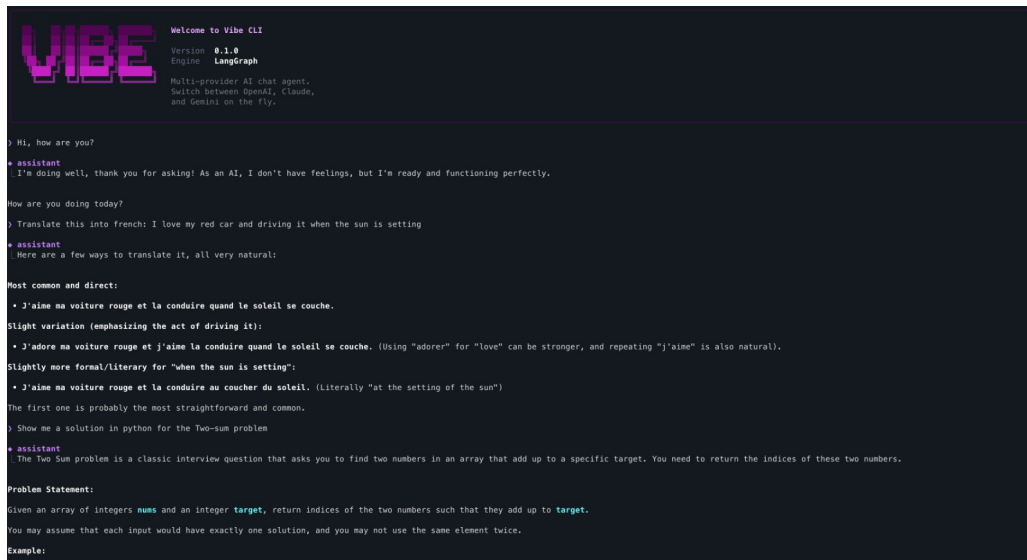
    Returns:
        The root node of the inverted binary tree.
    """
    # Send a message...

Use '/help' for more information
```

Figura 2.3: Continuarea conversatiei cu prima versiune a interfetei, realizata cu ajutorul libreriei Textual

In final, a fost o decizie buna sa abandonez implementarea cu Textual, deoarece am reusit sa produc rezultate mai bune folosind terminalul clasic. Solutia finala este si mai estetica, cat si simpla, functionala si performanta.

Schimbarea nu s-a produs intr-un stadiu avansat al dezvoltarii, ci chiar de la primele prototipuri de interfata, lucru care nu a ingreunat cu mult procesul de dezvoltare.



```

VIBE Welcome to Vibe CLI
Version 0.1.0
Engine LangGraph
Multi-provider AI chat agent.
Switch between OpenAI, Claude,
and Gemini on the fly.

> Hi, how are you?
• assistant
I'm doing well, thank you for asking! As an AI, I don't have feelings, but I'm ready and functioning perfectly.

How are you doing today?
> Translate this into french: I love my red car and driving it when the sun is setting
• assistant
Here are a few ways to translate it, all very natural:

Most common and direct:
• J'aime ma voiture rouge et la conduire quand le soleil se couche.

Slight variation (emphasizing the act of driving it):
• J'adore ma voiture rouge et j'aime la conduire quand le soleil se couche. (Using "adorer" for "love" can be stronger, and repeating "j'aime" is also natural).

Slightly more formal/literary for "when the sun is setting":
• J'aime ma voiture rouge et la conduire au coucher du soleil. (Literally "at the setting of the sun")

The first one is probably the most straightforward and common.

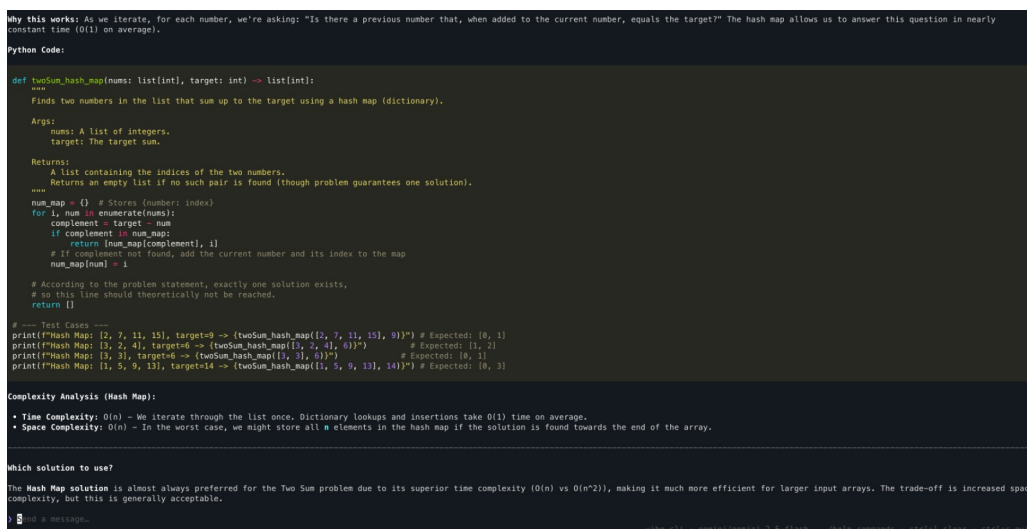
> Show me a solution in python for the Two-sum problem
• assistant
The Two Sum problem is a classic interview question that asks you to find two numbers in an array that add up to a specific target. You need to return the indices of these two numbers.

Problem Statement:
Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.
You may assume that each input would have exactly one solution, and you may not use the same element twice.

Example:

```

Figura 2.4: A doua versiune a interfetei, realizata fara libraria Textual



```

why this works: As we iterate, for each number, we're asking: "Is there a previous number that, when added to the current number, equals the target?" The hash map allows us to answer this question in nearly constant time O(1) on average.

Python Code:

def twoSum_hash_map(nums: list[int], target: int) -> list[int]:
    """
    Finds two numbers in the list that sum up to the target using a hash map (dictionary).

    Args:
        nums: A list of integers.
        target: The target sum.

    Returns:
        A list containing the indices of the two numbers.
        Returns an empty list if no such pair is found (though problem guarantees one solution).
    """
    num_map = {} # Stores (number: index)
    for i, num in enumerate(nums):
        complement = target - num
        if complement in num_map:
            return [num_map[complement], i]
        # If complement not found, add the current number and its index to the map
        num_map[num] = i

    # According to the problem statement, exactly one solution exists.
    # so this line should theoretically not be reached.
    return []

# --- Test Cases ---
print("Hash Map: [2, 7, 11, 13], target=9 -> [twoSum_hash_map([2, 7, 11, 13], 9)]") # Expected: [0, 1]
print("Hash Map: [1, 2, 4], target=6 -> [twoSum_hash_map([1, 2, 4], 6)]") # Expected: [1, 2]
print("Hash Map: [3, 3], target=6 -> [twoSum_hash_map([3, 3], 6)]") # Expected: [0, 1]
print("Hash Map: [1, 5, 9, 13], target=14 -> [twoSum_hash_map([1, 5, 9, 13], 14)]") # Expected: [0, 3]

Complexity Analysis (Hash Map):
• Time Complexity: O(n) - We iterate through the list once. Dictionary lookups and insertions take O(1) time on average.
• Space Complexity: O(n) - In the worst case, we might store all n elements in the hash map if the solution is found towards the end of the array.

Which solution to use?
The Hash Map solution is almost always preferred for the Two Sum problem due to its superior time complexity O(n) vs O(n^2)), making it much more efficient for larger input arrays. The trade-off is increased space complexity, but this is generally acceptable.

> Send a message.

```

Figura 2.5: Continuarea conversatiei cu a doua versiune a interfetei, realizata fara libraria Textual

Dupa inlocuirea implementarii, am continuat sa adaug functionalitati pentru a imbogati experienta utilizatorului.

Printre aceste functionalitati se numara:

- Alegerea modelului si a efortului de gandire pentru fiecare prompt
- Gestionarea conversatiilor, cu posibilitatea de a salva, sterge sau continua conversatii
- Conectarea cu servere MCP
- Utilizarea unor skill-uri specializate
- Consultarea contextului sau a consumului de tokens

Pentru utilizarea aplicatiei, utilizatorul trebuie sa ruleze comanda `make ui` in terminal, asigurandu-se ca a configurat corect programul sau facand-o direct din TUI daca a uitat sa o faca in prealabil. Dupa rularea comenzii, utilizatorul va fi intampinat de un meniu de start care ii va oferi optiunea de a incepe o conversatie noua sau de a continua una veche, precum si alte optiuni de configurare. Dupa selectarea uneia dintre aceste optiuni, utilizatorul va putea interactiona cu asistentul pentru a primi ajutor in sarcinile de dezvoltare software.

Comenzile pot fi rulate in orice moment al conversatiei, si au formatul `/comanda`. De asemenea exista si scurtaturi rapide pe tastatura, precum `CTRL+L` pentru a sterge conversatia si a incepe una noua, si `CTRL+C` pentru a inchide aplicatia.

Comanda	Descriere
<code>/help</code>	Comanda pentru listarea paginii de ajutor
<code>/model</code>	Selectarea modelului si introducerea cheii API
<code>/effort</code>	Selectarea nivelului de efort al modelului
<code>/mcp</code>	Meniul de configurare a serverelor Model Context Protocol (MCP)
<code>/skill</code>	Selectarea skill-urilor
<code>/plan</code>	Selectarea modului de planificare
<code>/agent</code>	Selectarea modului agentic
<code>/clear</code>	Stergerea conversatiei curente
<code>/exit</code>	Inchiderea aplicatiei
<code>/resume</code>	Continuarea unei conversatii anterioare

Tabela 2.2: Comenzi disponibile in interfata

BIBLIOGRAFIE

- [1] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A Survey of Large Language Models, March 2026.
- [2] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?, November 2025.
- [3] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 1958.
- [4] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, November 1998.
- [5] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9:1735–1780, November 1997.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners, July 2020.
- [8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35:24824–24837, December 2022.
- [9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*, September 2022.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.